МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп'ютерних технологій

ЗВІТ З ЛАБОРАТОРНОЇ

№4

на тему **"Рекурсія, мемоізація та динамічне програмування у Python"**

Варіант 15

Виконала:
Студентка 2 курсу
групи ФЕП-22
Сулятицька Христина

Перевірив:
Аспірант
Остріков Данило Андрійович

Львів-2026

Мета роботи. Ознайомитися з рекурсивними алгоритмами та обмеженнями рекурсії в Python, навчитися застосовувати мемоізацію за допомогою functools.lru_cache та динамічне програмування. Реалізувати розв'язання задачі про рюкзак 0/1 трьома способами (наївна рекурсія, рекурсія з мемоізацією, ітеративне ДП), створити тести коректності та порівняти час їх виконання.

Теоретичні відомості:

Рекурсія — це спосіб організації обчислень, за якого функція викликає саму себе для розв'язання менших підзадач. Рекурсивна функція обов'язково має базовий випадок, який завершує виклики без подальшої рекурсії, та рекурсивний крок, що зводить задачу до простішої. Без досяжного базового випадку виклики не припиняться.

Якщо одні й ті самі підзадачі розв'язуються багаторазово, наївна рекурсія може мати експоненційну складність. Мемоізація усуває повторні обчислення: результат функції для певних аргументів зберігається в кеші та повертається під час наступних викликів. У Python для цього призначений декоратор functools.lru_cache, а методи cache_info() і cache_clear() дають змогу переглянути статистику кешу та очистити його. Аргументи кешованої функції мають бути хешованими.

Динамічне програмування (ДП) будує розв'язок із розв'язків підзадач без рекурсії, зазвичай заповнюючи таблицю або масив. Ітеративна реалізація не використовує стек викликів, тому не залежить від обмеження глибини рекурсії.

CPython обмежує глибину рекурсії; поточне значення можна переглянути через sys.getrecursionlimit() (типово 1000). Перевищення ліміту призводить до винятку RecursionError. CPython не виконує оптимізацію хвостових викликів (TCO), тому навіть хвостова рекурсія використовує стек викликів.

Хід роботи

Для 15-го варіанта реалізовано задачу про рюкзак 0/1. Задано набір предметів, кожен з яких описується парою (вага, цінність), та місткість рюкзака. Потрібно знайти максимальну сумарну цінність предметів, вага яких не перевищує місткість; кожен предмет можна взяти не більше одного разу.

1. Створення структури проєкту

Проєкт розділено на модулі відповідно до їх призначення. У algorithms.py розміщено всі три реалізації алгоритму та перевірку вхідних даних. У main.py реалізовано консольне меню. Автоматизовані перевірки розміщено в каталозі tests.

Файл	**Призначення**
algorithms.py	Наївна рекурсія, мемоізований розв'язувач KnapsackSolver, ітеративне ДП, перевірка вхідних даних.

main.py	Інтерактивне консольне меню.
tests/test_knapsack.py	Перевірки коректності, кешу, вхідних даних та відсутності мутації.
tests/test_performance.py	Порівняння часу виконання трьох реалізацій.
benchmark_results.json	Збережені результати вимірювання часу.
test_results.txt	Результат запуску pytest.
requirements.txt	Залежності для тестування та перевірки якості коду.
pyproject.toml, mypy.ini	Налаштування black, ruff та mypy.
README.md	Опис задачі, алгоритмів, складності та запуску.

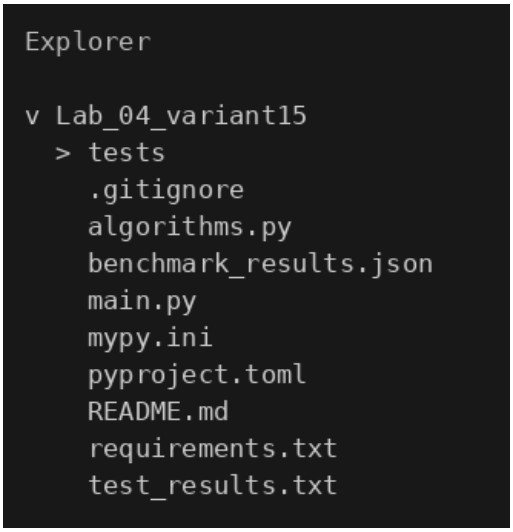


Рисунок 1 — Структура проєкту

2. Реалізація алгоритмів у файлі algorithms.py

Предмет задається типом Item = tuple[int, int]. Функція _validate() перевіряє, що місткість є цілим невід’ємним числом, кожен предмет — парою цілих чисел, вага додатна, а цінність невід’ємна; значення типу bool відхиляються. Некоректні дані викликають TypeError або ValueError.

Базовий випадок: усі предмети розглянуто або вільна місткість дорівнює нулю — повертається 0. Рекурсивний крок: для поточного предмета порівнюються два варіанти — не брати предмет або взяти його (якщо вага не перевищує залишкову місткість) — і повертається більша цінність. Функція knapsack_recursive() реалізує цю схему без кешування.

```python
"""Рекурсивні та ітеративні алгоритми для задачі про рюкзак 0/1."""
from __future__ import annotations

from functools import lru_cache
from typing import Protocol, TypeAlias

Item: TypeAlias = tuple[int, int]  # (вага, цінність)


class CacheStatistics(Protocol):
    """Поля статистики, які повертає functools.lru_cache."""

    @property
    def hits(self) -> int: ...

    @property
    def misses(self) -> int: ...

    @property
    def maxsize(self) -> int | None: ...

    @property
    def currsize(self) -> int: ...


def _validate(items: tuple[Item, ...], capacity: int) -> None:
    """Перевіряє вхідні дані та не допускає неоднозначних значень."""
    if isinstance(capacity, bool) or not isinstance(capacity, int):
        raise TypeError("Місткість рюкзака має бути цілим числом.")
    if capacity < 0:
        raise ValueError("Місткість рюкзака не може бути від'ємною.")
    for index, item in enumerate(items):
        if not isinstance(item, tuple) or len(item) != 2:
            raise TypeError(f"Предмет {index} має бути парою (вага, цінність).")
        weight, value = item
        if isinstance(weight, bool) or not isinstance(weight, int):
            raise TypeError("Вага предмета має бути цілим числом.")
        if isinstance(value, bool) or not isinstance(value, int):
            raise TypeError("Цінність предмета має бути цілим числом.")
        if weight <= 0:
            raise ValueError("Вага кожного предмета має бути більшою за нуль.")
        if value < 0:
            raise ValueError("Цінність предмета не може бути від'ємною.")


def knapsack_recursive(items: tuple[Item, ...], capacity: int) -> int:
    """Наївне рекурсивне розв'язання без мемоізації."""
    normalized = tuple(items)
    _validate(normalized, capacity)

    def solve(index: int, remaining: int) -> int:
        if index == len(normalized) or remaining == 0:
            return 0
        weight, value = normalized[index]
        if weight > remaining:
            return solve(index + 1, remaining)
        return max(
            solve(index + 1, remaining),
            value + solve(index + 1, remaining - weight),
        )

    return solve(0, capacity)
```

Рисунок 2 — Перевірка даних та наївне рекурсивне розв’язання

Клас KnapsackSolver містить метод _solve(index, remaining), декорований @lru_cache(maxsize=None). Стан підзадачі визначається парою (index, remaining), тому кожен стан обчислюється один раз. Метод solve() запускає розв’язання, а cache_info() і cache_clear() надають доступ до статистики та очищення кешу. Функція knapsack_memoized() є зручною обгорткою.

Функція knapsack_iterative() використовує одновимірний масив dp розміром capacity + 1. Місткість перебирається у зворотному порядку, щоб кожен предмет був використаний не більше одного разу.

```
65  class KnapsackSolver:
66      """Мемоізований розв'язувач; ключ кешу — (index, capacity)."""
67
68      def __init__(self, items: tuple[Item, ...], capacity: int) -> None:
69          self.items = tuple(items)
70          _validate(self.items, capacity)
71          self.capacity = capacity
72
73      @lru_cache(maxsize=None)
74      def _solve(self, index: int, remaining: int) -> int:
75          if index == len(self.items) or remaining == 0:
76              return 0
77          weight, value = self.items[index]
78          if weight > remaining:
79              return self._solve(index + 1, remaining)
80          return max(
81              self._solve(index + 1, remaining),
82              value + self._solve(index + 1, remaining - weight),
83          )
84
85      def solve(self) -> int:
86          """Повертає максимальну сумарну цінність предметів."""
87          return self._solve(0, self.capacity)
88
89      def cache_info(self) -> CacheStatistics:
90          """Повертає статистику кешу functools.lru_cache."""
91          return self._solve.cache_info()
92
93      def cache_clear(self) -> None:
94          """Очищає кеш мемоізованих станів."""
95          self._solve.cache_clear()
96
97
98  def knapsack_memoized(items: tuple[Item, ...], capacity: int) -> int:
99      """Зручна функція-обгортка для мемоізованого розв'язання."""
100     return KnapsackSolver(tuple(items), capacity).solve()
101
102
103 def knapsack_iterative(items: tuple[Item, ...], capacity: int) -> int:
104     """Ітеративне DP-рішення з одновимірним масивом."""
105     normalized = tuple(items)
106     _validate(normalized, capacity)
107     dp = [0] * (capacity + 1)
108     for weight, value in normalized:
109         for current_capacity in range(capacity, weight - 1, -1):
110             dp[current_capacity] = max(
111                 dp[current_capacity],
112                 value + dp[current_capacity - weight],
113             )
114     return dp[capacity]
115
116
117 def example_items() -> tuple[Item, ...]:
118     """Повертає набір предметів для демонстрації."""
119     return ((2, 12), (1, 10), (3, 20), (2, 15), (4, 25))
```

Рисунок 3 — Мемоізований розв’язувач та ітеративне ДП

3. Мемоізація та статистика кешу

Для демонстраційного набору предметів ((2, 12), (1, 10), (3, 20), (2, 15), (4, 25)) і місткості 5 усі три реалізації повертають 37 (предмети з вагами 1, 2 і 2 та цінностями 10, 15 і 12). Після одного виклику solve() на новому розв’язувачі cache_info() показує hits=5, misses=25, currsize=25: 25 унікальних станів обчислено, 5 разів результат узято з кешу.

Слід враховувати, що декоратор застосовано до методу, тому кеш існує на рівні класу, а self входить до ключа кешу. Звідси два наслідки: стани різних екземплярів не змішуються, але статистика cache_info() накопичується між екземплярами, а cache_clear() очищує кеш усіх екземплярів одночасно. Крім того, кеш зберігає посилання на екземпляри, тому вони не звільняються збирачем сміття, доки кеш не очищено.

4. Реалізація консольної оболонки у файлі main.py

Після запуску створюється розв’язувач для демонстраційного набору предметів і місткості 5. Меню дозволяє показати предмети, розв’язати задачу рекурсивно без кешу, з lru_cache або ітеративно, переглянути статистику кешу, очистити його та завершити роботу. Невідомий пункт меню обробляється повідомленням, після чого програма продовжує роботу.

```
=== Задача про рюкзак 0/1 — варіант 15 ===
1. Показати предмети та місткість
2. Розв'язати рекурсивно без кешу
3. Розв'язати рекурсивно з lru_cache
4. Розв'язати ітеративно
5. Показати статистику кешу
6. Очистити кеш
0. Вихід
Ваш вибір: 1

Предмети (номер: вага, цінність):
1: вага = 2, цінність = 12
2: вага = 1, цінність = 10
3: вага = 3, цінність = 20
4: вага = 2, цінність = 15
5: вага = 4, цінність = 25
Місткість рюкзака: 5

Ваш вибір: 3
Максимальна цінність: 37

Ваш вибір: 4
Максимальна цінність: 37

Ваш вибір: 5
Статистика кешу: CacheInfo(hits=5, misses=25, maxsize=None, currsize=25)

Ваш вибір: 0
Роботу завершено.
```

Рисунок 4 — Робота консольного меню (повторні виведення меню пропущено)

5. Створення та виконання тестів

Для перевірки реалізації створено автоматизовані тести pytest (файл tests/test_knapsack.py та tests/test_performance.py). Вони перевіряють такі властивості:

- усі три реалізації дають однакові результати на шести наборах даних, включно з порожнім набором, нульовою місткістю та випадком, коли жоден предмет не вміщується;
- кожен предмет використовується не більше одного разу;
- мемоізація заповнює кеш (misses > 0, currsize > 0), а cache_clear() скидає статистику;
- вхідні дані не змінюються під час роботи алгоритмів;
- некоректна місткість (-1, 1.5, True) та некоректні предмети (нульова або від'ємна вага, від'ємна цінність, неправильна структура) відхиляються винятками;
- порівняльний тест вимірює час трьох реалізацій і перевіряє збіг результатів наївного та ітеративного алгоритмів.

Під час контрольного запуску pytest усі 18 тестів завершилися успішно (результат збережено у файлі test_results.txt).

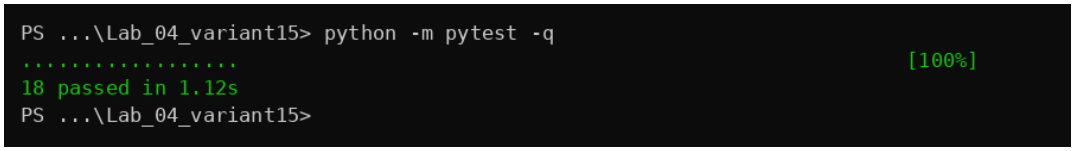


Рисунок 5 — Результат виконання тестів

6. Порівняння продуктивності

Для порівняння створено tests/test_performance.py. Вимірювання виконується за допомогою timeit на наборі з 22 предметів і місткості 35; кожна реалізація запускається 3 рази. Для мемоізованої версії в кожному запуску створюється новий розв'язувач, тобто вимірюється робота з порожнім кешем. Результати збережено у benchmark_results.json.

Реалізація	**Час виконання**
Наївна рекурсія	0,948382 с
Рекурсія з lru_cache	0,000952 с
Ітеративне ДП	0,000120 с

За результатами контрольного запуску наївна рекурсія виявилася приблизно в 1000 разів повільнішою за версію з lru_cache та приблизно в 7900 разів повільнішою за ітеративну. Мемоізована реалізація поступається ітеративній приблизно у 8 разів через накладні витрати на виклики функцій і роботу кешу. Наведені значення є результатом конкретного запуску й можуть відрізнятися залежно від обладнання та версії Python.

7. Обмеження глибини рекурсії

У використаному середовищі sys.getrecursionlimit() повертає 1000. Додатковий експеримент (поза автоматичними тестами) із 1500 предметами ваги 1 і цінності 1 та місткістю 1500 показав, що рекурсивна реалізація завершується винятком RecursionError (maximum recursion depth exceeded), тоді як ітеративна коректно повертає 1500. Це підтверджує, що для глибоких задач безпечніше застосовувати ітеративний алгоритм. Мемоізація скорочує кількість обчислень, але не зменшує глибину рекурсії.

8. Перевірка стилю та типів

У проєкті додано requirements.txt із pytest, mypy, black і ruff. У файлах pyproject.toml та mypy.ini задано налаштування інструментів (довжина рядка 88, Python 3.11, режим strict для mypy). Для локальної перевірки передбачено команди:

```
mypy .
black --check .
ruff check .
pytest -q
```

Під час підготовки звіту підтверджено успішне проходження 18 тестів pytest. Перевірки mypy, Black і Ruff необхідно виконати в локальному середовищі після встановлення відповідних пакетів; результат їх виконання в цьому звіті не позначається як підтверджений.

Висновок:

У ході лабораторної роботи я реалізувала розв’язання задачі про рюкзак 0/1 згідно з 15-м варіантом трьома способами: наївною рекурсією, рекурсією з мемоізацією через functools.lru_cache та ітеративним динамічним програмуванням з одновимірним масивом. Для кожного способу визначено базовий випадок і рекурсивний крок або схему оновлення масиву, додано перевірку вхідних даних і консольне меню зі статистикою кешу.

Автоматизовані тести pytest (18 тестів) підтвердили збіг результатів усіх реалізацій, коректну роботу кешу та відхилення некоректних даних. Порівняння часу показало, що наївна рекурсія з експоненційною складністю O(2^n) значно поступається мемоізованій та ітеративній реалізаціям зі складністю O(nW); найшвидшою виявилась ітеративна. Також я розглянула обмеження глибини рекурсії в CPython та відсутність оптимізації хвостових викликів і переконалася, що для великих вхідних даних ітеративний підхід надійніший.

Таким чином, я закріпила навички застосування рекурсії, мемоізації та динамічного програмування в Python і навчилася оцінювати їх ефективність та обмеження.